



Respuesta a la Propuesta de Integración V4

De: Grupo 8 — Tema 06, Sandbox / Runtime **Para:** Grupo 5 — Tema 05, Desafíos Prácticos **Sobre:** `Propuesta_Integracion_G5_G6_Entrypoint_V4.pdf` **Fecha:** 7 de septiembre de 2026

1. Respuesta corta

Aceptamos el modelo de dos capas de su §3, y elegimos la Opción 1 (catálogo de perfiles).

El §3 describe la arquitectura correcta, y la describe por las razones correctas: el `ENTRYPOINT` es de nosotros porque es lo único que puede manejar el nonce sin exponerlo, y la capa de evaluación es de ustedes porque es lo único que sabe qué significa "aprobar" en cada tipo de desafío. Es exactamente la separación que habíamos llegado a necesitar de nuestro lado, y coincide sin retoques.

Dicho eso, hay una asimetría que nos toca corregir a nosotros: **ustedes escribieron el V4 conociendo sólo la mitad de nuestro diseño.** Les habíamos pasado el mecanismo del nonce y la entrada del tar por `stdin`, pero no el resto del contrato del contenedor — dónde se dejan los resultados, cómo se avisa el estado, qué hace nuestro script *después* de que el de ustedes termina. Por eso su capa 1 tiene descrita la mitad de arriba y casi nada de la de abajo: no es un desacuerdo, es información que no les habíamos dado.

Este documento es esa mitad que faltaba. La sección §2 es la que importa: **es todo lo que un autor de scripts de evaluación necesita saber, y no es largo.**

2. El contrato de la capa 1, completo

Adentro del contenedor hay dos programas escritos por dos grupos que no se hablan entre sí. Corren uno después del otro. No hay HTTP, no hay llamadas a funciones, no hay parámetros: lo único que comparten es **el disco del contenedor y un número al final.**

Entonces todo el contrato cabe en las tres preguntas que se van a hacer cuando se sienten a escribir el script:

1. ¿Qué me voy a encontrar cuando arranque?
2. ¿Dónde tengo que dejar lo que produzca?
3. ¿Cómo aviso cómo me fue?

2.1 Qué se encuentra el script cuando arranca

En el momento exacto en que nuestra capa 1 lo invoca, el disco está así:

```

/work/          ← lo ÚNICO escribible. Vive en RAM, muere con el contenedor
├── in/          ← acá extrajimos el tar. El script arranca parado en esta carpeta
│   ├── run.sh   el script de evaluación
│   ├── src/...  el código del alumno
│   └── test/...  los archivos que hayan mandado
├── reports/     ← vacía. Es el buzón de salida
├── tmp/         ← vacía. Borrador, no vuelve
└── status/      ← vacía. Avisos opcionales (§2.3)

/libs/         ← junit.jar, archunit.jar, pmd... SÓLO LECTURA, viene en la imagen
/              ← todo el resto del disco: SÓLO LECTURA

```

Para que no tengan que escribir esas rutas a mano, se las pasamos también como variables de entorno:

Variable	Carpeta	Para qué
SANDBOX_IN	/work/in	dónde está el código; es el directorio de trabajo
SANDBOX_REPORTS	/work/reports	dónde dejar lo que tiene que volver
SANDBOX_TMP	/work/tmp	borradores
SANDBOX_STATUS	/work/status	avisos opcionales
SANDBOX_LIBS	/libs	dónde están las herramientas
SANDBOX_MEM_MB	ej. 512	cuánta memoria hay, para dimensionar la JVM

Estas variables las lee también el código del alumno, y está bien: **no son secretas**. Saber que los reportes van a `/work/reports` no le sirve de nada a quien quiere hacer trampa. Esa asimetría es justamente por lo que el nonce viaja por otro lado y estas no.

2.2 Dónde deja lo que produce

Una sola regla:

Todo lo que quede en `/work/reports` vuelve. Todo lo demás se pierde.

Empaquetamos esa carpeta entera y la mandamos. **No la miramos, no la interpretamos, no sabemos qué hay adentro:** si es un XML de JUnit, bien; si es uno de PMD, también; si son tres archivos, van los tres. Esa parte de nuestro entrypoint ya está construida y ya es agnóstica.

Y el corolario, que conviene tener presente al escribir el script: **si esa carpeta queda vacía, no hay aprobación posible**. Sin evidencia no hay veredicto.

Este punto sí cambia algo de su diseño

El §3 del V4 dice que la capa 2 "devuelve un JSON con la nota". Ese JSON no puede salir por `stdout`.

El motivo: nuestra capa 1 **desvía `stdout` y `stderr` del script a archivos** antes de invocarlo, y arma el sobre de respuesta después, cuando ya no hay código de nadie corriendo. Eso es lo que garantiza que nada de lo que imprima el alumno pueda entrar al sobre — las marcas con el nonce impiden *falsificar* el reporte, pero si `stdout` quedara vivo no impedirían *contaminarlo*.

Las salidas del script y del alumno **no se pierden**: vuelven adentro del sobre, en su propio campo y truncadas a un tope. Simplemente no son el canal del resultado.

El JSON con la nota va como un archivo más en `/work/reports`.

2.3 Cómo avisa cómo le fue

Con el código con el que termina. Y con una aclaración importante:

El número no es el veredicto. El veredicto sale del reporte, siempre. El número sólo dice **qué tan lejos llegó el script**.

Proponemos repartir el espacio así:

Rango	Dueño	Significado
0	ustedes	"llegué hasta el final, mirá el reporte"
40-59	ustedes	"me frené a propósito, y este número dice por qué" — la tabla la definen ustedes
20-31	nosotros	reservados; son los que ya usa nuestro endpoint hoy
32-39	—	hueco a propósito, para que podamos crecer sin renegociar
cualquier otro	—	se traduce a error de infraestructura

El motivo de las bandas: un código de salida es **un byte**, y buena parte ya tiene dueño por convención de Unix. El 0 es universal; el 1 y el 2 los devuelven `javac`, `java`, PMD y el propio `sh` ante cualquier error; el 126 es "no lo puedo ejecutar"; el 127 es "no encontré el comando"; y 128+N es "murió por una señal" (137 = sin memoria, 152 = se acabó la CPU). Si usaran 1 para "no compila", no lo podríamos distinguir de "el shell se rompió".

Opcionalmente, si quieren que el alumno vea "no compila" en vez de un error genérico, pueden dejar un archivo `$SANDBOX_STATUS/fase` con una etiqueta (`COMPILACION`, `PRUEBAS`, `ANALISIS` ...) y otro `$SANDBOX_STATUS/detalle` con una línea de diagnóstico. Los copiamos al sobre tal cual.

¿Por qué las dos cosas, el número y el archivo? Porque **el archivo no siempre llega, y el número sí**. Si el script se muere de golpe —lo mata el kernel por consumo de CPU, tiene un error de sintaxis— no alcanzó a escribir nada. El número igual nos llega, porque lo produce el sistema operativo. El archivo es el canal rico pero frágil; el número, el canal pobre pero indestructible.

2.4 Qué hace nuestra capa 1, completa

Para que quede la película entera. Los pasos en **negrita** son los que no estaban en el §3 del V4:

Antes de invocar el script:

1. Lee el nonce de la primera línea de `stdin`, en una variable de shell que **no se exporta** (si se exportara, el alumno la leería de `/proc/1/environ` aunque después la borráramos).
2. Lee los bytes del tar del mismo `stdin`.
3. **Valida el tar antes de extraerlo**: rutas relativas, sin `..`, sin barra inicial, y **sin enlaces simbólicos ni duros**. Un tar con un symlink que apunte afuera de `/work` es la familia de bug que tumbó a Judge0 tres veces, y es entrada que nos llega desde su API — así que también conviene que la rechacen ustedes antes de armarlo. Doble validación, a propósito.
4. Extrae en `/work/in` y prepara `reports/`, `tmp/` y `status/`.
5. **Desvía `stdout` y `stderr` a archivos** (§2.2).
6. Imprime la marca de inicio y hace `cd /work/in && sh ./run.sh`.

Después de que el script termina:

1. **Barre y cuenta los procesos que sobrevivieron al script**. El vector es concreto: el código del alumno deja un proceso en segundo plano que sobrevive al runner y reescribe el reporte con su propio veredicto. No se puede prevenir —el reporte está en una carpeta que el alumno puede escribir— pero **sí se puede detectar**: si sobrevivió algún proceso, la entrega no puede terminar en éxito.
2. **Empaqueta `/work/reports` y trunca las salidas** a un tope de bytes.
3. **Clasifica y emite el sobre**, rodeado por las marcas con el nonce.

Los pasos 7, 8 y 9 son la razón por la que el `ENTRYPOINT` tiene que ser nuestro y tiene que recuperar el control **después** de la evaluación, no sólo antes. Un entrypoint que sólo abre y cierra marcas alrededor de la llamada deja los tres afuera.

3. Sobre las tres opciones

Elegimos la **Opción 1**. Pero antes, una observación que les puede resultar útil:

A nivel del contenedor, **las Opciones 1 y 3 son la misma opción**. En la Opción 1 el script vive en nuestra base y lo inyectamos en el tar; en la Opción 3 lo inyectan ustedes en el tar. En los dos casos, lo que la capa 1 hace es idéntico: `cd /work/in && sh ./run.sh`. Y la Opción 2 se expresa como un `run.sh` generado a partir del array de comandos.

O sea que **un solo mecanismo del lado nuestro sirve para las tres**, y podemos empezar a construir sin que la elección quede bloqueada. La decisión entre ellas no es técnica del contenedor: es de dónde se guarda el script y quién lo versiona. Ahí la Opción 1 gana clarísimo, por lo que sigue.

4. El catálogo de perfiles: contrapropuesta al endpoint

El endpoint del §5 es la idea correcta. Le proponemos cinco ajustes, y cuatro son chicos.

4.1 Perfiles inmutables y versionados

El `PUT /profiles/{id}` del V4 hace que el perfil sea **modificable en el lugar**. El problema no es de seguridad, es de trazabilidad: si mañana editan `java21-junit`, dos ejecuciones con el mismo `profileId` se comportan distinto y no hay forma de saber cuál corrió con qué. No podríamos recorrer la entrega de un alumno de hace tres semanas ni responder *"¿con qué se corrigió esto?"*.

Propuesta: `POST /profiles` crea una versión nueva y devuelve `{profileId, version, hash}`; modificar una versión existente se rechaza. Cada ejecución guarda la versión y el hash del script que corrió. Para ustedes el flujo de trabajo no cambia —siguen publicando sin depender de un despliegue nuestro—, y a cambio la reproducibilidad queda garantizada.

Con ciclo de vida `BORRADOR → VALIDADA → ACTIVA → DEPRECADA`. Deprecar una versión no rompe nada en vuelo, porque cada ejecución apunta a la suya.

4.2 `image` sale de un catálogo, no es texto libre

El §6 del V4 dice —y estamos de acuerdo— que las imágenes base las construimos nosotros. Siendo así, también las nombramos nosotros: el perfil referencia un `imagenId` de nuestro catálogo, y una imagen desconocida es un `422` al registrar el perfil, no un intento a ciegas en ejecución.

Es el mismo criterio de siempre: **nadie de afuera fija los recursos del sandbox**, ni siquiera indirectamente eligiendo la imagen.

4.3 Los límites viven en el perfil, no en el request

En el payload del §5, `limits.timeoutMs` viaja con cada ejecución. Preferimos moverlo al perfil, y el motivo es que **el perfil es justamente la cosa que sabe si esto es una corrida de JUnit o un análisis de PMD**, que necesitan presupuestos distintos. Mandarlo por ejecución obliga a que quien arma el request sepa algo que el perfil ya sabe.

Y nos permite algo que de otro modo se pierde: **declarar dos presupuestos separados, compilación y evaluación**. Nos importa porque no queremos cobrarle al alumno el tiempo de compilar —no depende de él, y en nuestras mediciones es la mitad del tiempo total. Sin esa separación, un alumno con una solución correcta puede irse a timeout por culpa del compilador.

Una nota técnica que puede sorprender: **el reloj del alumno lo medimos en tiempo de CPU, no de reloj de pared**. Con reloj de pared, dos entregas idénticas dan resultados distintos según cuán cargado esté el host —medimos entre 1,8 y 5,2 segundos para el mismo bundle. En CPU es estable. Si nos mandan un `timeoutMs`, lo tomamos como sugerencia y lo recortamos contra el techo del perfil.

4.4 Autenticación

Es un endpoint de escritura que cambia qué código corre adentro de nuestros contenedores. Va con autenticación servicio a servicio. Lo mencionamos sólo para que no quede implícito.

4.5 Un cambio de nombre

`entrypointScript` conviene que se llame `evalScript` o simplemente `script`. No es el entrypoint —el entrypoint es la capa 1 y es nuestro—; es la capa 2. El nombre reinstala justo la confusión que su §3 resuelve.

4.6 Dos campos que les proponemos agregar

```
{
  "name": "Java 21 - JUnit Runner",
  "imagenId": "sandbox-java-21-tools",
  "script": "#!/bin/sh\n...",
  "reportFormat": "junit-xml",
  "exitCodes": { "41": "No compila la solución", "42": "No compilan las pruebas" },
  "limits": { "compileMs": 20000, "evalCpuS": 10, "memoriaMb": 512 }
}
```

- **reportFormat** nos dice cómo leer lo que quedó en el buzón. Lo necesitamos para una verificación que hoy hace nuestro entrypoint y que bajo este esquema se muda afuera del contenedor: **comprobar que el reporte contiene evidencia real de ejecución**. Nace de algo que medimos: un `System.exit(0)` del alumno hacía que el runner devolviera éxito. Desde entonces la regla es que el veredicto sale del reporte y nunca del código de salida — y para aplicarla hay que saber qué formato tiene el reporte.
- **exitCodes** es opcional y cuesta nada: es su tabla de la banda `40-59`, y nos permite mostrarle al alumno *"no compila"* en vez de un error genérico.

4.7 Sobre la auditoría de los scripts

El V4 ofrece como ventaja para nosotros el poder auditar los scripts antes de correrlos. Se lo agradecemos, pero preferimos ser francos: **no la necesitamos como control de seguridad, y no queremos prometer una revisión que en la práctica nadie va a hacer con rigor**.

El script es capa 2, y la capa 2 ya está tratada como no confiable: corre sin red, sin root, con `/libs` de sólo lectura, con límites de CPU y memoria, encerrada entre dos tapas que no controla y sin acceso al nonce. **Un `run.sh` mal escrito o malicioso no puede hacer nada que el código del alumno no pudiera hacer ya**. Esa es la propiedad que hace que esta integración sea barata para los dos.

Lo que sí vamos a hacer al registrar un perfil es un **smoke test automático**: lo corremos una vez contra un par de bundles de referencia —una solución que sabemos buena y una que sabemos mala— y verificamos tres cosas: que deja algo en el buzón, que devuelve un código de su banda, y que los dos bundles no dan el mismo veredicto. Si pasa, el perfil queda **VALIDADA**.

Esto es una ventaja para ustedes, no un portón: les da en segundos el error que de otro modo aparecería recién con la entrega de un alumno.

5. Dos observaciones sobre el Dockerfile del \$6

El Dockerfile está bien encaminado —Alpine, usuario no-root, `/libs` con las herramientas— y esa es la base de la que vamos a partir. Dos cosas a corregir:

1. **`chown -R sandboxuser /libs` le da al alumno permiso de escritura sobre las herramientas**. El código del alumno corre como `sandboxuser`, así que podría reescribir `junit.jar` o el PMD **entre fases**: compilar bien, y reemplazar el jar del analizador antes de que corra. No rompe el aislamiento del contenedor, pero sí la integridad de las herramientas con las que se dicta el veredicto. `/libs` tiene que quedar de sólo lectura para el usuario del sandbox, igual que el entrypoint.
2. **Falta `/work`**. El Dockerfile tiene `WORKDIR /app`, pero la carpeta escribible tiene que ser `/work`, y va montada

como **tmpfs** —vive en RAM y muere con el contenedor— con el resto del sistema de archivos en sólo lectura. Es lo que hace que no quede nada entre ejecuciones sin depender de que alguien limpie.

Los dos ajustes los aplicamos nosotros al construir las imágenes; los mencionamos para que el Dockerfile del documento no quede como referencia.

6. Lo que falta en el V4 y necesitamos cerrar

El modelo asincrónico. El V4 no lo menciona, y es una parte del contrato que importa tanto como el resto: **la ejecución no responde en línea**. Nuestra API acepta el pedido con `202 Accepted` y un identificador, y el resultado llega después.

La vía natural es la que ya usa la plataforma: **el resultado viaja como evento por el bus**, el de Kafka que implementa el grupo de notificaciones. De nuestro lado publicamos `EjecucionFinalizada` apenas la ejecución termina:

```
EjecucionFinalizada {
  "eventId": "...", "tipo": "EjecucionFinalizada", "version": 1,
  "ocurridoEn": "...", "correlationId": "...",
  "payload": { "ejecucionId": "...", "entregaId": "...", "intentoNro": 2,
               "estado": "...", "resumen": { ... } }
}
```

El evento lleva el resumen, no el detalle. Quien necesite el reporte completo hace un `GET /api/v1/sandbox/ejecuciones/{id}`. Así el payload del bus se mantiene chico y no arrastramos reportes enteros por el bus de eventos de toda la plataforma.

Lo que necesitamos confirmar con ustedes es poco: que efectivamente lo consumen por ahí, cómo quieren que se llame el evento o el tópico, y si además del evento les sirve el `GET` como respaldo para el caso en que se pierdan un mensaje. Nada de eso bloquea el resto del contrato.

Junto con eso, dos cosas que ya estaban acordadas y conviene dejar escritas de nuevo, porque bajo este esquema las calculan ustedes y nosotros sólo les damos el insumo: **un error de infraestructura no consume intento del alumno**, y **una suite que no compila tampoco** — si los tests del profesor están rotos, la culpa no es de quien entregó.

7. Qué necesitamos de ustedes

Ordenado por cuánto pesa. Casi todo lo de este documento lo resolvemos de nuestro lado — el contrato de la §2 vive en nuestra capa 1 y funciona sin que ustedes hagan nada. Lo que sigue es lo que **no** podemos resolver solos.

Lo único que les cambia el diseño

#	Qué	Por qué
1	El JSON de la nota va a <code>/work/reports</code> , no a <code>stdout</code> (§2.2)	Es el único punto del V4 que no podemos cubrir desde nuestra capa: su capa 2 tiene <code>stdout</code> desviado a un archivo, así que un JSON impreso ahí no llega como resultado — se mezcla con lo que imprimió el alumno y se trunca

Dos decisiones que tomamos juntos

#	Qué	Por qué
2	Confirmación de la Opción 1 , con perfiles versionados e inmutables (§4.1)	Desbloquea que construyamos el catálogo. El refactor del endpoint no espera esto (§3), pero el CRUD sí
3	Que consumen el resultado por el bus de eventos , y cómo se llama el evento (§6)	Es la vía que ya usa la plataforma; sólo hay que fijar el nombre y decidir si quieren el <code>GET</code> como respaldo

Cuatro insumos que sólo ustedes pueden dar

Ninguno bloquea el contrato: sin ellos el sistema funciona, sólo que peor. Los pedimos ahora porque cuanto antes lleguen, mejor arranca.

#	Qué	Qué pasa si no llega
4	La tabla de la banda 40-59 (§2.3)	Funciona igual, pero el alumno ve <i>"detenido por la evaluación"</i> en vez de <i>"no compila"</i>
5	reportFormat de cada perfil (§4.6)	Nos quedamos sólo con la regla gruesa: buzón vacío, no hay éxito. Perdemos la verificación fina de evidencia
6	Los presupuestos de compilación y evaluación por perfil (§4.3)	Usamos un techo único holgado. La distorsión es chica, pero compilar deja de ser gratis para el alumno
7	La lista de herramientas para /libs , con versiones	Sólo podemos ofrecer los perfiles que ya tengamos armados. Es el pedido que más conviene adelantar: el contenedor no tiene red, así que nada que no esté en la imagen se puede usar

De nuestro lado arrancamos ya con el refactor del endpoint al modelo de dos capas, porque como señalamos en §3 es el mismo trabajo en las tres opciones y no depende de la respuesta a este documento.

Resumen

Coincidimos en lo importante: el `ENTRYPOINT` es de G8, la evaluación es de G5, y el nonce nunca cruza de una capa a la otra. Lo que este documento agrega es la mitad del contrato que todavía no les habíamos pasado — el buzón de salida, los códigos de salida, el desvío de `stdout` y lo que la capa 1 hace después de que su script termina.

De todo eso, **una sola cosa les cambia el diseño**: el JSON con la nota va a `/work/reports`, no a `stdout`. El resto son ajustes al endpoint de perfiles y dos correcciones al Dockerfile.